

NAME – TAMANNA GURAV

ROLL NO – 44

UID – 225072

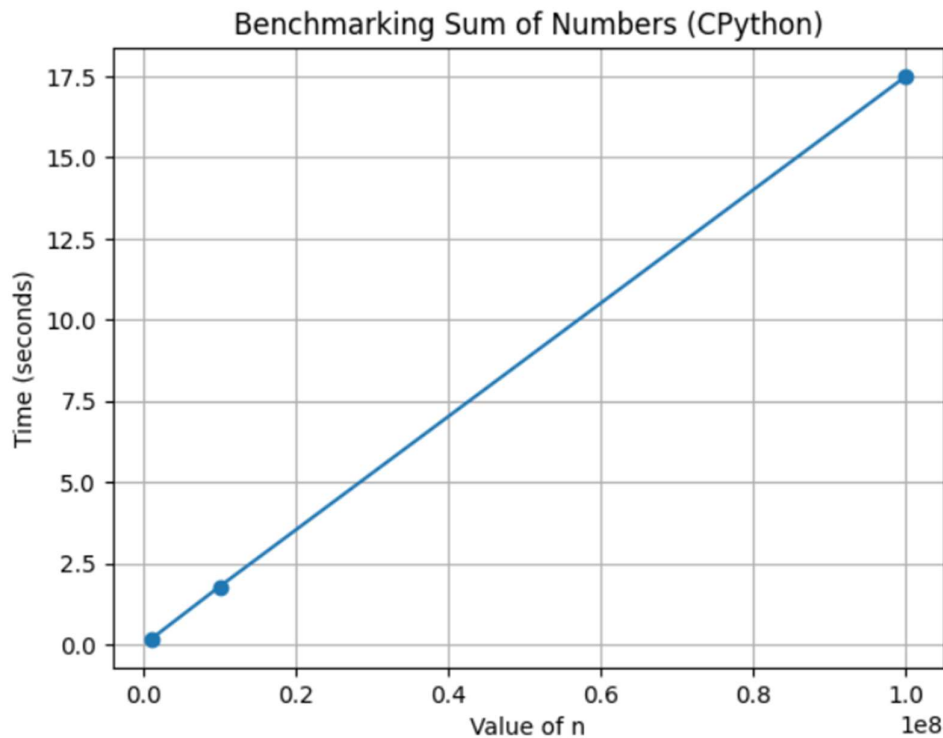
PERFORMANCE ANALYSIS(CPython)

Benchmarking Algorithm :

```
def sum_numbers(n):  
    return sum(range(n))  
  
# Benchmarking function for CPython performance  
Codeium: Refactor | Explain | Generate Docstring | ✕  
def benchmark_sum(n):  
    start_time = time.time() # Record the start time  
    sum_numbers(n) # Perform the summing  
    return time.time() - start_time # Return the time taken  
  
# Parallel summing function (using multiprocessing)  
Codeium: Refactor | Explain | Generate Docstring | ✕  
def parallel_sum(start, end):  
    return sum(range(start, end))
```

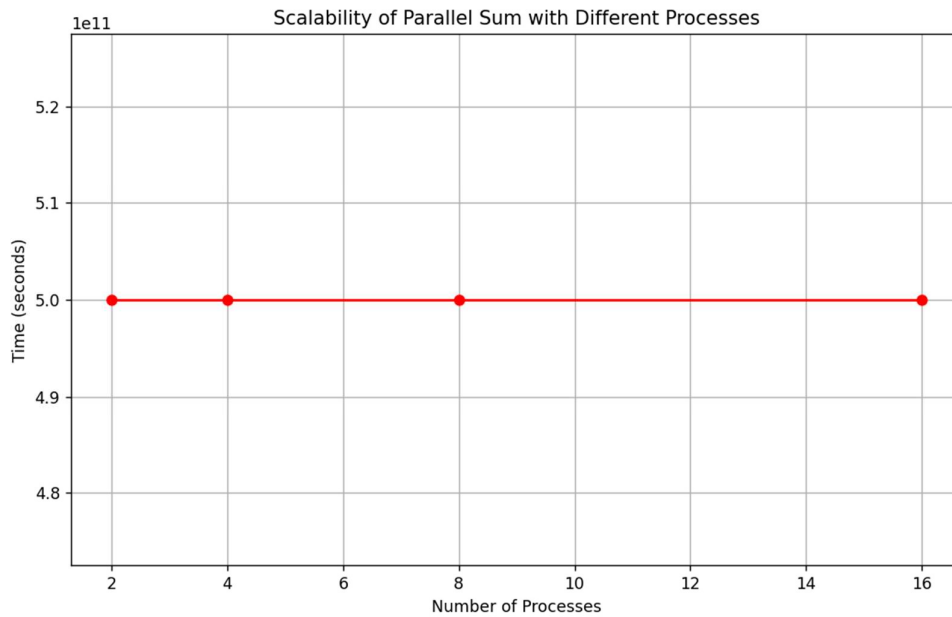
- The benchmarking algorithm you implemented calculates the time taken to sum numbers up to a given n using CPython.
- The process is as follows:
- Functionality: The `sum_numbers(n)` function sums all numbers from 0 to n-1 using Python's built-in `sum` function and `range()`.
- Benchmarking: The `benchmark_sum(n)` function records the time before and after calling the summing function, and then computes the time difference to evaluate how long it takes to perform the summing operation.
- Test Scenarios: The benchmarking was carried out for different values of n, such as 1 million (1,000,000), 10 million (10,000,000), and 100 million (100,000,000), to demonstrate how time increases as the size of the dataset grows.

Chart For Benchmarking Sum of Numbers Using CPython



- X-Axis: The chart plots the value of n (the size of the range) against the time taken for the summing operation.
- Y-Axis: The time taken in seconds to sum numbers from 0 to $n-1$.
- Insight: As the value of n increases, the time taken to sum the numbers also increases. This illustrates the linear relationship between the size of the dataset and the time it takes to process it.

SCALABILITY



- The `parallel_sum(start, end)` function divides the range into chunks and assigns each chunk to a separate process. The `benchmark_parallel_sum()` function aggregates the results from all processes.
- If the task is small or not computationally expensive, the overhead of starting and managing multiple processes can outweigh the benefits of parallel execution. The overhead involved in creating processes, dividing the workload, and aggregating the results can lead to minimal improvement in execution time or even a slight increase in time.

Conclusion

- **CPython Performance:** The performance of CPython in summing numbers increases linearly with larger datasets.
- **Parallelization Benefits:** The parallelization with multiprocessing shows that for large datasets, using multiple processes can significantly improve performance, but the benefit starts to diminish as the number of processes increases beyond a certain point.
- **Scalability:** This experiment validates that Python's multiprocessing can be used to optimize computation-heavy tasks, but there are diminishing returns due to process management overheads and system resource limitations.